Experiment-8

Aim: Construct a Binary Search Tree and implement Tree Traversals.

1. Binary Search Tree (BST)

Definition:

A **Binary Search Tree** is a binary tree in which each node has at most two children and satisfies the following property:

- The **left subtree** of a node contains nodes with values **less than** the node's value.
- The **right subtree** of a node contains nodes with values **greater than** the node's value.
- Both left and right subtrees must also be BSTs.

Applications:

- Searching and sorting efficiently
- Implementing sets and maps
- Hierarchical data storage

2. Operations in BST

1. **Insert:** Add a node maintaining the BST property.
2. **Search:** Find a node by value.
3. **Traversals:** Visit all nodes in a specific order:
    - **Inorder (L → Root → R):** Gives sorted order
    - **Preorder (Root → L → R):** Used to copy tree
    - **Postorder (L → R → Root):** Used to delete tree

3. Algorithm for Insertion

1. If tree is empty, create a new node as root.
2. Compare value to current node:
    - If value < current node → move to left child.
    - If value > current node → move to right child.
3. Repeat until the correct position is found.
4. Insert the new node.

Program:

```
# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

Dept of AIML

```
# BST class
class BST:
    def __init__(self):
        self.root = None

    # Insert method
    def insert(self, data):
        if self.root is None:
            self.root = Node(data)
        else:
            self._insert(self.root, data)

    def _insert(self, current, data):
        if data < current.data:
            if current.left is None:
                current.left = Node(data)
            else:
                self._insert(current.left, data)
        elif data > current.data:
            if current.right is None:
                current.right = Node(data)
            else:
                self._insert(current.right, data)
        # Ignore duplicate values

    # Inorder Traversal
    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    # Preorder Traversal
    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    # Postorder Traversal
    def postorder(self, node):
```

Dept of AIML

```
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

# Example Usage
bst = BST()
values = [50, 30, 70, 20, 40, 60, 80]
for val in values:
    bst.insert(val)

print("Inorder Traversal (sorted):")
bst.inorder(bst.root)
print("\nPreorder Traversal:")
bst.preorder(bst.root)
print("\nPostorder Traversal:")
bst.postorder(bst.root)
```

OUTPUT:
Inorder Traversal (sorted):
20 30 40 50 60 70 80
Preorder Traversal:
50 30 20 40 70 60 80
Postorder Traversal:
20 40 30 60 80 70 50

Dept of AIML